PROJECT REPORT

Payment processing simulation in Node.js microservices

The brief asked for one Node.js service that creates payments, retrieves them and updates their status, with simulated asynchronous processing. This project delivers that as a payment service, and adds an auth service and a notification service that talk to it over NATS JetStream. Every planned milestone is done, the README was verified from a clean clone, and the stretch phase is deferred.

Repository: github.com/VICTOR-CHUKWU/payment-simulation · State of `main` on 19 October 2026 · Built 18 to 19 October 2026

3 + 1
services plus a shared library, in one pnpm monorepo

36
milestones done across 6 phases (2 merged into others, 4 stretch deferred)

11
REST endpoints, each in its service's OpenAPI spec with Swagger UI

873
unit and API tests in 62 suites, none needing Docker

99.5%
line coverage (96.6% of branches)

6
cross-service integration tests over real NATS and SMTP

The brief and where it is met

Each requirement from the assessment, and the part of the project that answers it.

REQUIREMENT	WHERE IT IS MET
Express server with create (POST), retrieve by ID (GET) and update status (PATCH) endpoints	`POST /payments`, `GET /payments/:id` and `PATCH /payments/:id/status` in the payment service
Simulated payment processing with asynchronous operations	A background processor sends each new payment to a simulated gateway that approves, declines or times out after a random delay
In-memory or JSON-file persistence	A repository interface with both adapters, picked by `STORAGE_DRIVER`. File writes are atomic
Proper error responses, with emphasis on error handling and logging	RFC 9457 Problem Details from one central error model; pino JSON logs with a correlation ID that follows a payment across services; retries and a dead-letter queue for email
Unit tests (Jest)	873 unit and API tests, plus an integration suite

REQUIREMENT	WHERE IT IS MET
API documentation (Swagger or Markdown)	Swagger UI and an OpenAPI 3.1 spec per service, generated from the Zod schemas, and endpoint tables in the README
Git repository with a README covering setup, API and testing	Public repository; README checked step by step from a fresh clone

What was added, and why

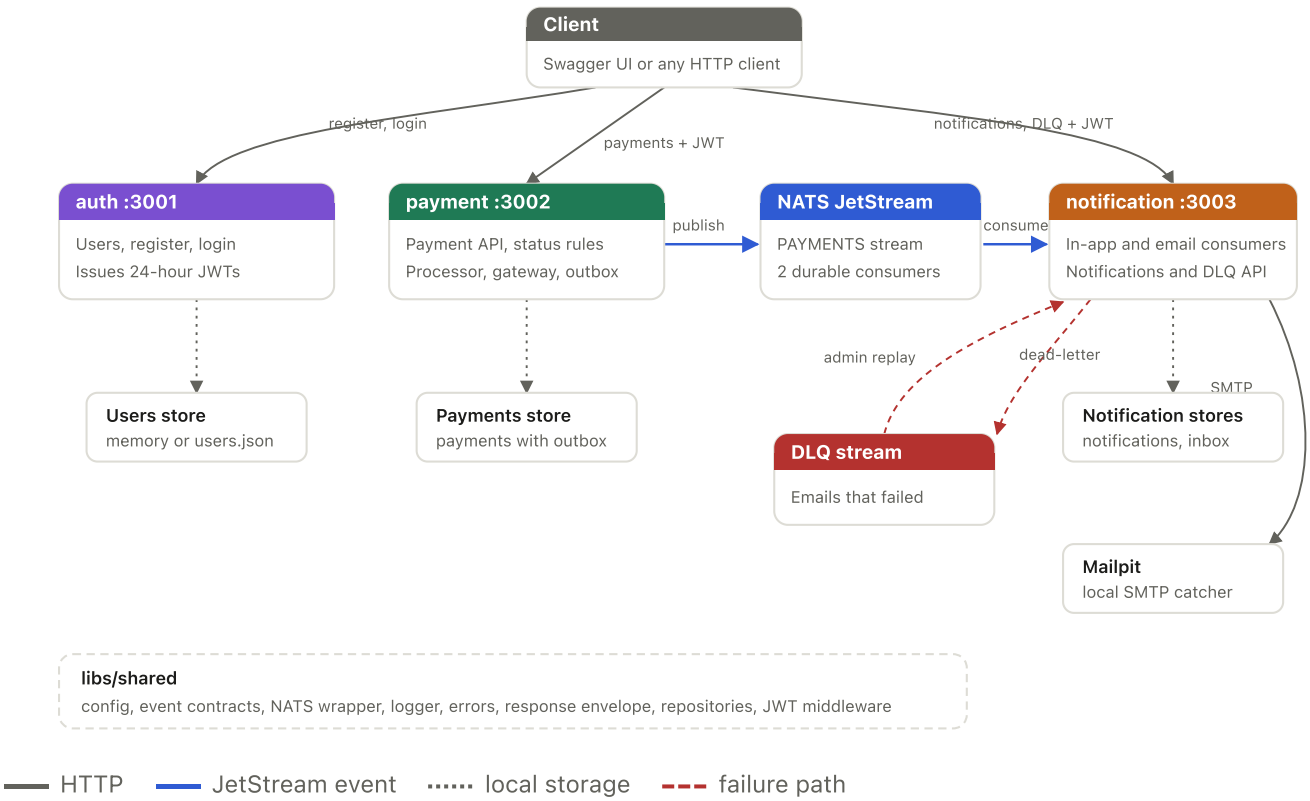
Auth service. It gives every payment an owner. Users only see their own payments, and only an admin can change a payment's status. Without users, neither rule has anything to check.

Notification service. It gives the payment events a consumer, which makes the messaging between services real. It stores in-app notifications and sends emails, and it handles duplicate deliveries and failed emails.

Everything else stays as simple as the brief asks: storage is in memory or a JSON file, there is no database, and auth covers only register and login.

Architecture

Each service owns its data and never calls another service over HTTP. Payment events flow through JetStream, and every service checks JWTs itself with a shared secret, so nothing calls auth after login.



COMPONENT	OWNS	TALKS TO
`services/auth`	Users and bcrypt password hashes. Issues HS256 JWTs carrying `sub`, `email` and `role`. Seeds one admin and one user	Nothing at runtime
`services/payment`	Payments, their status rules, and the outbox of events not yet published	NATS: publishes to `PAYMENTS`
`services/notification`	In-app notifications, and per channel the events already handled (the inbox)	NATS: consumes `PAYMENTS`, writes and reads `DLQ`. SMTP
`libs/shared`	No data. Code every service needs, so services never import each other	None

How a payment moves through the system

One ID follows the payment from the HTTP request through the processor to both notification channels, so one log

search shows the whole path.

1. **The user logs in.** Auth checks the password with bcrypt and returns a JWT with the user's ID, email and role.
   auth
2. **The user creates a payment.** `POST /payments` validates the body with Zod and honours an optional `Idempotency-Key`. It saves the payment as `pending` and a `payment.created` event in one write, and returns `201`.
   payment
3. **The outbox relay publishes the event** to JetStream, with the event ID as `Nats-Msg-Id` so a repeated publish is dropped. If NATS is down, the event waits.
   payment
4. **The processor settles the payment.** It claims it (`processing`), calls the simulated gateway, and records `succeeded` or `failed` with a compare-and-set update and the matching event.
   payment
5. **Notification receives each event twice,** once per channel through its own durable consumer. Each channel skips an event it has already handled.
   notification
6. **The user is told.** The in-app channel stores a notification for `GET /notifications`. The email channel sends a message through SMTP, which Mailpit catches locally.
   notification

An admin can cancel a `pending` payment or settle a `processing` one with `PATCH /payments/:id/status`. Final states are `succeeded`, `failed` and `cancelled`; any other change returns `409 invalid-status-transition`.

Failure handling

The failures the design plans for, and what happens when each one occurs.

FAILURE	WHAT HAPPENS
Invalid request	Zod rejects it, and the central error model returns `400` Problem Details pointing at each bad field.
Client retries a payment request	The same `Idempotency-Key` returns the existing payment instead of creating a second one.
Admin and processor change the same payment	Both use compare-and-set on `version`. The first write wins; the admin gets `409` or the processor logs `payment already settled`.
NATS down, or a crash before publishing	The event waits in the payment's outbox and is published on the next sweep or at startup. One payment's events stay in order.

FAILURE	WHAT HAPPENS
Same event delivered twice	JetStream drops a repeat publish within 2 minutes. After that, each channel's inbox skips it.
Email send fails temporarily	The message is nak'd with a growing delay (5 s, 30 s, 2 min, 10 min), up to `EMAIL_MAX_ATTEMPTS` deliveries.
Email rejected for good, or attempts run out	The handler publishes it to the `DLQ` stream and acks it. An admin lists it and replays it through `/admin/dlq`.
Invalid event on the stream	It is terminated, so it never loops, and a warning is logged without its contents.
Crash during a JSON-file write	Writes go to a temporary file renamed into place, with one writer per file, so a file is never half-written.
Shutdown with payments in flight	They stay `processing` and are resumed on the next start.
A dependency goes away	`/health/ready` returns `503` until NATS, storage or the consumers recover.

API

Every protected endpoint needs `Authorization: Bearer <token>`. Success responses share one envelope (`status`, `success`, `data`, `message`, `meta` for pages); errors use RFC 9457 Problem Details. The full reference is each service's Swagger UI at `/docs`.

SERVICE	METHOD	PATH	WHO	WHAT IT DOES
auth	POST	`/auth/register`	anyone	Creates a `user` account
auth	POST	`/auth/login`	anyone	Returns a 24-hour JWT; rate limited per IP
payment	POST	`/payments`	user, admin	Creates a `pending` payment
payment	GET	`/payments/:id`	owner, admin	Reads one payment
payment	PATCH	`/payments/:id/status`	admin	Cancels or settles a payment
notification	GET	`/notifications`	owner	Lists own notifications, paginated, unread filter
notification	PATCH	`/notifications/:id/read`	owner	Marks one as read

SERVICE	METHOD	PATH	WHO	WHAT IT DOES
notification	PATCH	`/notifications/read`	owner	Marks all as read
notification	GET	`/admin/dlq`	admin	Lists dead-lettered emails
notification	GET	`/admin/dlq/:seq`	admin	Reads one entry
notification	POST	`/admin/dlq/:seq/replay`	admin	Sends the email again and deletes the entry

Each service also serves `/health/live`, `/health/ready`, and in development `/docs` with `/docs/openapi.json`.

What was delivered

The work ran as 42 milestones in 7 phases, each with a task doc in `tasks/`, its own tests, and a pull request: 30 merged in all. Milestone IDs never change once assigned, so a few rows sit out of number order.

Phase 1 Foundation 10 done

BE-01-01	Monorepo scaffold: pnpm workspaces, TypeScript project references, Biome with `noConsole` as an error	done
BE-01-03	Local NATS JetStream and Mailpit through docker-compose and `infra:*` scripts	done
BE-01-02	Shared library: config loader, event contracts, NATS client wrapper	done
BE-01-04	Service skeleton: liveness, readiness on real dependencies, graceful shutdown	done
BE-01-05	Structured logging with redaction and correlation IDs	done
BE-01-06	Central error model (Problem Details)	done
BE-01-07	Repository interface with compare-and-set, and the in-memory adapter	done
BE-01-08	JSON-file adapter with atomic, single-writer writes	done
BE-01-09	OpenAPI from Zod schemas, Swagger UI at `/docs`	done
BE-01-10	Standard success response envelope	done

Phase 2 Users and auth 4 done

BE-02-01	Auth service with user model and seeded admin and user accounts	done

BE-02-02	Register and login with bcrypt and JWT	done
BE-02-03	Shared JWT verification and role guard	done
BE-02-04	HTTP hardening: helmet, CORS, login rate limit	done

Phase 3 Payments 7 done

BE-03-01	Payment model (owner, minor-unit amount and currency, version) and status state machine	done
BE-03-02	`POST /payments` with validation	done
BE-03-07	Idempotency keys on payment creation	done
BE-03-03	`GET /payments/:id` with ownership	done
BE-03-04	`PATCH /payments/:id/status`, transitions enforced atomically	done
BE-03-06	Status PATCH is admin-only	done
BE-03-05	Simulated gateway and background processor, resuming after restart	done

Phase 4 Messaging 2 done

BE-04-01	JetStream streams, subjects and event catalogue	done
BE-04-02	Lifecycle events published through an outbox stored with the payment	done

Phase 5 Notification service 6 done, 1 cancelled

BE-05-01	Notification service with separate durable consumers for in-app and email	done
BE-05-02	Idempotent handling, deduplicated per channel	done
BE-05-03	In-app notifications: store, list own, mark as read	done
BE-05-04	Email channel over SMTP with a template per event	done
BE-05-05	Email retries with backoff	done
BE-05-06	Dead-letter stream with admin inspect and replay	done

BE-05-07	Ownership and role checks, split into BE-03-06 and BE-05-03	cancelled

Phase 6 Quality and deliverables 7 done

BE-06-01	Unit test gaps closed, plus a coverage report	done
BE-06-02	API tests for every documented status of every endpoint	done
BE-06-03	Cross-service integration tests	done
BE-06-04	OpenAPI completeness: examples, error responses and auth on every operation	done
BE-06-05	Final README	done
BE-06-06	Architecture docs with diagrams	done
BE-06-07	README verified from a clean clone of the public repository	done

Phase 7 Stretch deferred

BE-07-01	Idempotency keys, moved to BE-03-07	cancelled
BE-07-02	`GET /payments` list with pagination and status filter	deferred
BE-07-03	Dockerfiles and a full-stack compose file	deferred
BE-07-04	GitHub Actions CI	deferred
BE-07-05	Prometheus metrics per service	deferred

Key decisions

The choices that shaped the build. The full list is the Decisions table in `ROADMAP.md`, and each task doc records the reasoning for its own design.

Stack

Node.js 24, TypeScript 6, Express 5, pnpm workspaces. Biome through Ultracite for lint and format. Root pnpm scripts run everything, including Docker.

Messaging

NATS JetStream: durable streams, durable pull consumers, explicit acks. Every event has a unique ID and every consumer is idempotent.

Outbox and inbox

Events are saved with the status change and published afterwards. Each notification channel records what it handled, so redeliveries are skipped.

Persistence

A repository interface with in-memory and JSON-file adapters. No database or ORM. Every update is compare-and-set on a `version` field.

Auth

24-hour JWTs with `user` and `admin` roles, verified locally by each service. No refresh tokens, password reset or OAuth.

Money

Integer amounts in minor units with an ISO 4217 code. Floats are never used.

Validation and errors

Zod at every boundary: HTTP input, event payloads and env config. One error model returns RFC 9457 Problem Details.

Logging

pino JSON logs with a correlation ID carried in event headers. Passwords, tokens, auth headers and emails are redacted.

Left out on purpose

No cache, because data already lives in memory and each service runs as one instance. No tracing or metrics server.

Testing and verification

Every milestone shipped with its own tests. Phase 6 closed the remaining gaps and added a suite that runs all three services together.

COMMAND	WHAT IT RUNS	RESULT ON `MAIN`
`pnpm test`	Unit and API tests next to the code; API tests drive each app with supertest over in-memory storage	873 passed in 62 suites
`pnpm test:coverage`	The same, with a v8 coverage report	99.53% lines, 96.63% branches

COMMAND	WHAT IT RUNS	RESULT ON `MAIN`
`pnpm test:integration`	All three services as real processes against test-only NATS and Mailpit	6 passed, about 4 s
`pnpm check`, `pnpm typecheck`, `pnpm build`	Lint and format, TypeScript including tests, compiled build	Clean

- **OpenAPI tests** fail if any operation lacks a description, examples, its error responses or its auth rules, or if an example does not match its schema.
- **The integration suite** covers a payment reaching its owner in-app and by email with one correlation ID; temporary SMTP failures retried and dead-lettered; permanent rejections dead-lettered at once; DLQ replay; and the same event delivered twice. It found and fixed one real bug: a temporary `451` recipient refusal was being treated as permanent.
- **The clean-clone check** followed the README word for word from a fresh clone: install, start infrastructure and services, log in as both demo accounts, create a payment, see it settle with two notifications and two emails, and run every test command. No README fix was needed.

Known limitations

Deliberate trade-offs for a simulation, each recorded in its task doc.

A gateway timeout marks the payment `failed`. Safe here because the simulated gateway never charges on a call it hangs on. A real system would treat a timeout as unknown and check with the gateway first.	BE-03-05
An event NATS rejected for good would block that payment's later events, because the relay keeps one payment's events in order. No such error is known.	BE-04-02
A redelivery that never reached the send still uses an email attempt, for example when the inbox file cannot be read.	BE-05-05
An email can be sent twice after a crash between sending and recording it, with the same `Message-ID`. Recording first would risk losing the email instead.	BE-05-02
Under `pnpm dev` and `pnpm start`, one Ctrl+C reaches each service twice, and the second signal force-exits before shutdown finishes. Running `node dist/main.js` directly is not affected.	BE-01-05

Running it

Needs Node.js 24, pnpm (the version pinned in `package.json`) and Docker with Compose.

Start everything

```
pnpm install
cp .env.example .env
pnpm infra:up
pnpm dev
```

`infra:up` starts NATS and Mailpit; `dev` serves ports 3001 to 3003. Demo accounts: `admin@example.com` / `Admin123!` and `user@example.com` / `User123!`. Emails appear in Mailpit at `localhost:8025`.

Test

```
pnpm test
pnpm test:coverage
pnpm infra:test:up
pnpm test:integration
pnpm infra:test:down
```

The integration suite uses its own containers, so it never touches dev data.

---

payment-simulation · MIT licence · Full detail in `README.md`, `docs/architecture.md`, `ROADMAP.md` and `tasks/`.